

Practica Patrón Observer

Víctor Manuel Peiró Martínez

FIIS 2ºA

Diseño De Software

Solución:

Información:

Observer:

El observador es el elemento que ha de ser informado de los cambios de estado en el observable. En este caso, nuestros observadores son los departamentos ya que cada vez que se devuelve un libro en mal estado han de estar informados. Cada observador está definido por la interfaz **PullPushObserver** y nuestros observadores son las clases **StockDept**, **AdminDept** y **ComprasDept**.

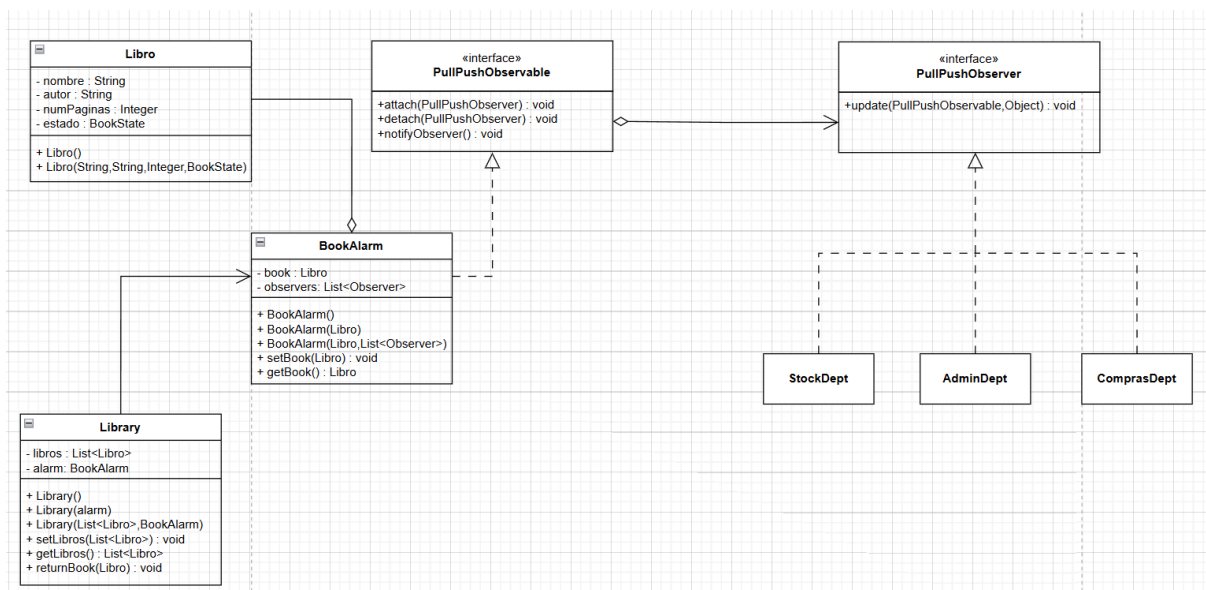
La interfaz contiene el método update que es el encargado de actualizar el estado. Al estar usando el protocolo pull/push, se puede notificar usando cualquiera de los protocolos. Pero tenemos que definir una prioridad de consumo de manera que si se recibe la variable de estado (Protocolo Push) lo consumirá directamente y si no pues lo coje del observable (Protocolo Pull).

Observable:

El observable es el elemento que ha de ser monitorizado en cada cambio de estado. En nuestro caso, la clase **BookAlarm** es nuestro observable ya que es la clase encargada de notificar si se quiere devolver un libro en mal estado. Esta clase contiene como atributos el libro que va a ser devuelto que es el estado del que vamos a notificar y la lista de observadores a los que notificar. La manera de notificar es mediante el setter. Una vez cambiado el estado, el setter llama a la función para notificar a los observadores del cambio de estado

El observable está definido por la interfaz **PullPushObservable**. Esta interfaz tiene 3 métodos que son triviales para todo observador. El método attach es el encargado de añadir un observador a la lista de observadores. El método detach es el encargado de eliminar un observador de la lista. Finalmente el método notifyObservers es el encargado de notificar a todos los observadores sobre el cambio de estado llamando a su método update.

UML:



Código:

Interfaces:

```
package EntregaObserverPattern;
//Interfaz plantilla para los observadores
public interface PullPushObserver {
    public void update(PullPushObservable observable, Object state);
}
```

```
package EntregaObserverPattern;
//Interfaz plantilla de los observable
public interface PullPushObservable {
    public void attach(PullPushObserver observer);
    public void detach(PullPushObserver observer);
    public void notifyObservers();
}
```

Observadores:

```
package EntregaObserverPattern;
//Observador
public class AdminDept implements PullPushObserver{
    public void update(PullPushObservable observable, Object state) {
        //Primero comprueba si ha recibido el estado
        //Protocol Push
        if (state instanceof Libro) {
            System.out.println("[AdminDept Push] se ha devuelto el libro en mal estado"
                               + (Libro)state);

            //Protocolo Pull
        } else if (observable instanceof BookAlarm) {
            System.out.println("[AdminDept Pull] se ha devuelto el libro en mal estado"
                               + ((BookAlarm)observable).getBook());
        }
    }
}

package EntregaObserverPattern;
//Observador
public class ComprasDept implements PullPushObserver{
    public void update(PullPushObservable observable, Object state) {
        //Primero comprueba si ha recibido el estado
        //Protocol Push
        if (state instanceof Libro) {
            System.out.println("[ComprasDept Push] se ha devuelto el libro en mal estado"
                               + (Libro)state);

            //Protocolo Pull
        } else if (observable instanceof BookAlarm) {
            System.out.println("[ComprasDept Pull] se ha devuelto el libro en mal estado"
                               + ((BookAlarm)observable).getBook());
        }
    }
}
```

```

package EntregaObserverPattern;
//Observador
public class StockDept implements PullPushObserver{
    public void update(PullPushObservable observable, Object state) {
        //Primero comprueba si ha recibido el estado
        //Protocol Push
        if (state instanceof Libro) {
            System.out.println("[StockDept Push] se ha devuelto el libro en mal estado"
                               + (Libro)state);

            //Protocolo Pull
        } else if (observable instanceof BookAlarm) {
            System.out.println("[StockDept Pull] se ha devuelto el libro en mal estado"
                               + ((BookAlarm)observable).getBook());
        }
    }
}

```

Observable:

```

package EntregaObserverPattern;

import java.util.ArrayList;
import java.util.List;

public class BookAlarm implements PullPushObservable{

    //Atributos
    private Libro book; //Estado del que se informa
    private List<PullPushObserver> observers;

    //Constructor sin parametros
    public BookAlarm() {
        this(new Libro());
    }

    //Constructor de 1 parametro
    public BookAlarm(Libro book) {
        this(book,new ArrayList<PullPushObserver>());
    }

    //Constructor principal
    public BookAlarm(Libro book, List<PullPushObserver> observers) {
        this.book = book;
        this.observers = observers;
    }

    //Getters y Setter del libros
    public Libro getBook() {

```

```

        return this.book;
    }

    public void setBook(Libro book) {
        this.book = book;
        this.notifyObservers(); //Al cambiar el estado, se notifica a todo observador
    }

    //Añade a un observador
    public void attach(PullPushObserver observer) {
        this.observers.add(observer);
    }

    //Elimina a un observador
    public void detach(PullPushObserver observer) {
        this.observers.remove(observer);
    }

    //Notifica de cambio de estado a todos los observadores
    public void notifyObservers() {
        for (PullPushObserver observer : observers) {
            observer.update(this, this.book);
        }
    }
}

```

Clase Libro y Relacionados:

```

package EntregaObserverPattern;
public class Libro {

    //Constantes
    public static final String DEFAULT_NOMBRE = "El Quijote De La Mancha";
    public static final String DEFAULT_AUTOR = "Miguel de Cervantes";
    public static final Integer DEFAULT_NUMPAGINAS = 844;
    public static final BookState DEFAULT_ESTADO = BookState.UNKNOWN;

    //Atributos
    private String nombre;
    private String autor;
    private Integer numPaginas;
    private BookState estado;

    //Constructor por defecto --> Crea el Quijote
    public Libro() {
        this(DEFAULT_NOMBRE, DEFAULT_AUTOR, DEFAULT_NUMPAGINAS,
            DEFAULT_ESTADO);
    }
}

```

```

//Constructor principal
public Libro (String nombre, String autor, Integer numPaginas, BookState estado) {
    this.nombre = nombre;
    this.autor = autor;
    this.numPaginas = numPaginas;
    this.estado = estado;
}

//Getters y Setters
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
public String getAutor() {
    return this.autor;
}
public void setAutor(String autor) {
    this.autor = autor;
}
public Integer getNumPaginas() {
    return this.numPaginas;
}
public void setNumPaginas(Integer numPaginas) {
    this.numPaginas = numPaginas;
}
public BookState getEstado() {
    return this.estado;
}
public void setEstado(BookState estado) {
    this.estado = estado;
}

//Metodo toString
public String toString() {
    return "Libro [nombre=" + this.nombre + ", autor=" + this.autor + ", numPaginas="
        + this.numPaginas + ", estado=" + this.estado + "]";
}
}

package EntregaObserverPattern;

//Enumerado con los posibles estados de un libro
public enum BookState {
    GOOD,BAD,UNKNOWN
}

```

Library:

```
package EntregaObserverPattern;
import java.util.ArrayList;
import java.util.List;
public class Library {
    //Atributos
    private List<Libro> libros;
    private BookAlarm alarm;

    //Constructor sin parametros
    public Library() {
        this(new BookAlarm());
    }
    //Constructor con 1 parametros
    public Library(BookAlarm alarm) {
        this(new ArrayList<Libro>(),alarm);
    }
    //Constructor con 2 parametros
    public Library (List<Libro> libros,BookAlarm alarm) {
        this.libros = libros;
        this.alarm = alarm;
    }
    //Devuelve libros en mal estado
    void returnBook(Libro book) {
        if (book.getEstado().equals(BookState.BAD)) {
            alarm.setBook(book);
        }
        this.libros.add(book);
    }

    //Getters y Setters
    public List<Libro> getLibros() {
        return this.libros;
    }

    public void setLibros(List<Libro> libros) {
        this.libros = libros;
    }

    public BookAlarm getAlarm() {
        return this.alarm;
    }

    public void setAlarm(BookAlarm alarm) {
        this.alarm = alarm;
    }
}
```

Test:

```
package EntregaObserverPattern;
public class LibraryTest {
    public static void main(String[] args) {
        //Crear una biblioteca
        Library library = new Library();

        //Creo los observadores y los añado a la lista de observadores
        StockDept stock = new StockDept();
        AdminDept admin = new AdminDept();
        ComprasDept compras = new ComprasDept();

        library.getAlarm().attach(stock);
        library.getAlarm().attach(admin);
        library.getAlarm().attach(compras);

        //Creo un libro en buen estado y en mal estado e intento devolverlos
        Libro quijote = new Libro();
        quijote.setEstado(BookState.BAD);

        Libro buenLibro = new Libro("Gang of 4 Design Pattern", "Erich Gamma, Richard Helm",
288, BookState.GOOD);

        System.out.println("Alarma notifica bajo protocolo PULL-PUSH");
        library.returnBook(buenLibro);
        library.returnBook(quijote);

        //Creo libro en mal estado e intento devolverlo
        System.out.println("Alarma vuelve a notificar bajo protocolo PULL-PUSH");
        Libro malLibro = new Libro("Programar sin patrones", "Golfito Martinez", 10,
BookState.BAD);

        library.returnBook(malLibro);
    }
}
```

Resultado:

```
<terminated> LibraryTest [Java Application] C:\Users\Victor Peiro\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin\javaw.exe (1 mar. 2025, 15:42:12 - 15:42:14)
Alarma notifica bajo protocolo PULL-PUSH
[StockDept Push] se ha devuelto el libro en mal estadoLibro [nombre=El Quijote De La Mancha, autor=Miguel de Cervantes, numPaginas=844, estado=BAD]
[AdminDept Push] se ha devuelto el libro en mal estadoLibro [nombre=El Quijote De La Mancha, autor=Miguel de Cervantes, numPaginas=844, estado=BAD]
[ComprasDept Push] se ha devuelto el libro en mal estadoLibro [nombre=El Quijote De La Mancha, autor=Miguel de Cervantes, numPaginas=844, estado=BAD]
Alarma vuelve a notificar bajo protocolo PULL-PUSH
[StockDept Push] se ha devuelto el libro en mal estadoLibro [nombre=Programar sin patrones, autor=Golfito Martinez, numPaginas=10, estado=BAD]
[AdminDept Push] se ha devuelto el libro en mal estadoLibro [nombre=Programar sin patrones, autor=Golfito Martinez, numPaginas=10, estado=BAD]
[ComprasDept Push] se ha devuelto el libro en mal estadoLibro [nombre=Programar sin patrones, autor=Golfito Martinez, numPaginas=10, estado=BAD]
```